

ABHISHEK TIWARI / PERSONAL REFERENCE

Shell and Linux Visual Notes

Command examples, Bash comparisons, and visual references from the class notes

Author	Abhishek Tiwari
--------	-----------------

Edition	September 2026
---------	----------------

Personal study notes edited into a public portfolio edition. Internal identities, private URLs, credentials, and client information are excluded.

Shell reference

These short text notes are paired with the original visual examples from the supplied class-notes archive.

Bash script

While "shell script" is a generic term, it often refers to scripts written for the POSIX shell (sh), whereas "bash scripts" are specifically written for the Bourne-Again Shell (bash). Here are key differences with code examples:

1. Arrays (Bash-only feature)

```
#!/bin/bash
# Bash script (works)
fruits=("apple" "banana" "cherry")
echo ${fruits[1]} # Outputs "banana"

# POSIX shell script (fails), check with sh
# Arrays are not supported in basic shell
```

2. String Manipulation

```
#!/bin/bash
str="hello world"
echo ${str^^} # Outputs "HELLO WORLD" (uppercase conversion)

# POSIX shell would need external commands like:
echo "$str" | tr '[:lower:]' '[:upper:]'
```

3. Parameter Expansion Features

```
#!/bin/bash
var="hello"
echo ${var:1:3} # Outputs "ell" (substring)

# POSIX shell would need:
echo "$var" | cut -c 2-4
```

4. Brace Expansion

```
#!/bin/bash
echo file.{txt,log} # Expands to "file.txt file.log"

# Not available in basic shell
```

5. Command History Features

only works in interactive shell sessions. It will not work in scripts.

```
!!      # Repeats last command
!$      # Last argument of previous command
```

6. Special Variables

```
# Bash script
echo ${BASH_VERSION} # Outputs bash version
# No equivalent in POSIX shell
```

Shell script

You can determine your shell type using the `ps` command

```
#!/bin/sh
# This is a comment!
echo Hello World      # This is a comment, too!
```

```
chmod 755 first.sh
./first.sh
```

```
#!/bin/sh
# This is a comment!
echo "Hello      World"      # This is a comment, too!
```

observe the space in output due to being just one argument

Variables

Note that there must be no spaces around the "=" sign
create `var.sh`

```
#!/bin/sh
MY_MESSAGE="Hello World"
echo $MY_MESSAGE
```

The shell does not care about types of variables; they may store strings, integers, real numbers - anything you like.

```
#!/bin/sh
echo What is your name?
read MY_NAME
echo "Hello $MY_NAME - hope you're well."
```

the shell-builtin command `read` which reads a line from standard input into the variable supplied.

Loops

we have `for` and `while` loops in the Bourne shell. This is somewhat fewer features than other languages, but nobody claimed that shell programming has the power of C.

```
#!/bin/sh
for i in 1 2 3 4 5
do
    echo "Looping ... number $i"
done
```

```
#!/bin/sh
INPUT_STRING=hello
while [ "$INPUT_STRING" != "bye" ]
do
    echo "Please type something in (bye to quit)"
    read INPUT_STRING
    echo "You typed: $INPUT_STRING"
done
```

The colon (:) always evaluates to true; whilst using this can be necessary sometimes,
it is often preferable to use a real exit condition

```
#!/bin/sh
while :
do
    echo "Please type something in (^C to quit)"
    read INPUT_STRING
    echo "You typed: $INPUT_STRING"
done
```

Test is most often invoked indirectly via the if and while statements.
if there isn't a space there, it won't work

```
echo "Enter value of x"
read x
if [ "$x" -eq "2" ]
then
    echo "you have put the right number"
else
    echo "you have put the wrong number"
fi
```

```
echo "Enter value of x"
read x

if [ "$x" -eq "5" ]; then
    echo "you have put the number 5"
elif [ "$x" -lt "5" ]; then
    echo "You have put less than 5 number"
elif [ "$x" -gt "5" ]; then
    echo "You have put greater than 5 number"
else
    echo "None of the above"
fi
```

Functions

```
echo "Enter the value"
```

```
read x
```

```
is_odd(){  
    if [ $((x%2)) == 0 ]; then  
        echo "Is Even Number"  
        exit 1  
    else  
        echo "Number is Odd"  
    fi  
}  
  
is_odd $x
```

Shell and Bash comparison

Here is a comparison of shell and Bash scripting with examples:
Shell Scripting (sh)

- Definition: Shell scripting refers to writing scripts using a command-line interpreter, often based on the Bourne shell (sh) or a POSIX-compliant shell.
- Portability: Shell scripts are designed to be highly portable across various Unix-like systems.
- Syntax: Uses basic syntax suitable for simple tasks.
- Features: Limited features compared to Bash, lacking advanced capabilities like associative arrays.

Example Shell Script (sh)

```
#!/bin/sh  
  
echo "Hello, this is a shell script."  
for i in 1 2 3; do  
    echo "Number: $i"  
done
```

Bash Scripting

- Definition: Bash scripting uses the Bourne Again Shell (Bash), an enhanced version of the original Bourne shell.
- Portability: Generally compatible with most Unix-like systems where Bash is installed.
- Syntax: Supports more advanced syntax and features.
- Features: Includes features such as associative arrays, command completion, process substitution, and more complex conditionals.

Example Bash Script

```
#!/bin/bash  
  
echo "Hello, this is a Bash script."  
for (( i=1; i<=3; i++ )); do  
    echo "Number: $i"  
done  
  
declare -A my_array  
my_array["key1"]="value1"  
my_array["key2"]="value2"  
  
echo "Array values: ${my_array[@]}"
```

Key Differences

- Features: Bash offers more advanced features, making it suitable for complex scripts.
- Portability: Shell scripts are more portable, while Bash scripts may have compatibility issues if they use Bash-specific features.
- Interactive Use: Bash is more user-friendly with command history, auto-completion, and inline editing.
- Speed: Shell scripts (especially those using dash) can be faster for system procedures.

When to Use Each

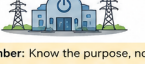
- Shell Scripting (sh): Ideal for simple, portable scripts that need to run on a wide range of Unix-like systems.
- Bash Scripting: Suitable for complex tasks requiring advanced features, automation, and user-friendly functionalities.

Both shell and Bash scripting are valuable tools for automating tasks, managing systems, and enhancing productivity in Unix-like environments.

AI responses may include mistakes.

Visual references

Linux filesystem reference

Linux File System = A City				
Everything in Linux starts from a single root directory (/).				
Directory	City Analogy	Purpose (What it stores)	Key Points	Examples / Commands
 /	 The city itself This is the main city, where everything exists.	The root of the entire file system. All directories and files branch out from here.	- Top-most directory - All files and directories are located somewhere under /	<pre>cd / ls /</pre> Lists all top-level directories in the city.
 /home	 Residential area Where citizens (users) live with their personal belongings.	Contains home directories for regular users. Each user gets their own space to store personal files.	- Example: /home/sameer - Users can read, write, and manage their own files here.	<pre>cd /home ls /home</pre> Shows all users' home directories.
 /etc	 Government offices (rules and policies) Where the city's rules, policies, and systems are managed.	Stores system-wide configuration files. These files control how the system and services behave.	- Critical system files - Editing files here can change system behavior - Example: /etc/passwd, /etc/hostname	<pre>ls /etc cat /etc/os-release</pre> View configuration files and system information.
 /bin	 Tool shops Provides essential tools that every citizen can use.	Contains essential user commands (binaries). These are needed for basic system operation.	- Commands used by all users - Must be available even in single-user mode - Examples: ls, cp, mv, cat	<pre>ls /bin which ls</pre> Lists basic commands available in the tool shop.
 /var	 Warehouses with changing inventory Stores items that keep changing every day.	Contains files whose content changes frequently. Examples: logs, databases, mail, spool files, caches.	- Logs are inside /var/log - Mail data in /var/mail - Can grow very large	<pre>ls /var ls /var/log</pre> View changing data and logs.
 /tmp	 Temporary storage area Used for short-term needs. Cleaned regularly.	Stores temporary files. Files in /tmp may be deleted automatically (on reboot or periodically).	- Anyone can create/delete files here - Do not store important data permanently	<pre>cd /tmp touch test.txt</pre> Create temporary files here.
 /dev	 Utility department Represents all hardware devices and connections of the city.	Contains device files. Linux treats devices (disks, USB, keyboard, terminals, etc.) as files.	- Examples: /dev/sda, /dev/tty, /dev/null - Do not edit or delete files inside	<pre>ls /dev lsblk</pre> Shows all available devices.
 /boot	 Power station startup controls Controls how the city starts and powers up.	Contains files needed to boot (start) the Linux system. Examples: kernel, initrd, bootloader (GRUB) files.	- Needed during system boot - Critical for startup - Do not delete files here	<pre>ls /boot</pre> View boot files and kernel.
💡 Remember: Know the purpose, not just the names. If you understand the role of each directory, navigating Linux becomes easy!				

Integer comparison example

Integer comparison

Operator	Description
-eq	is equal to
-ne	is not equal to
-gt	is greater than
-ge	is greater than or equal to
-lt	is less than
-le	is less than or equal to

String comparison example

String Comparison

Operator	Description
==	is equal to
!=	is not equal to
\<	is less than, in ASCII alphabetical order
\>	is greater than, in ASCII alphabetical order